MINIPROJECT ENHANCEMENT REPORT

Ticket Management System – Enhancement Report

Admin Dashboard Notifications Enhancement

Student Name	: Neeraj Gupta
Enrollment / ID	: 1032232796
Course / Department	: Computer Science Engineering / Computer Engineering & Technology
Institution Name	: Dr. Vishwanath Karad MIT World Peace University
Guide Name	: Dr. Suhas Joshi
Submission Date	: 27 April, 2026

ABSTRACT

This report documents the individual enhancement contribution of Neeraj Gupta to the Ticket Management System. The enhancement focuses on implementing Admin Dashboard Notifications — a real-time alerting mechanism that notifies administrators when new tickets are raised and when existing ticket statuses change. The feature ensures that administrators can respond to incoming tickets and status transitions promptly without manually refreshing the dashboard or querying the ticket list.

The implementation introduces a backend notification service that generates and stores notification records for relevant ticket events, a REST API endpoint to fetch unread notification counts and notification details, and a frontend notification panel integrated into the Admin dashboard. The design is modular and backward compatible, extending the existing system without modifying current ticket creation or status update flows.

ENHANCEMENT OVERVIEW

Background

The existing Ticket Management System provides administrators with a dashboard that lists tickets and their statuses. However, administrators must manually check the dashboard or refresh the page to discover newly raised tickets or status changes. This creates a lag in response time and increases the risk of missed or delayed ticket handling. The Admin Dashboard Notifications enhancement addresses this by introducing an event-driven alerting layer that proactively surfaces new activity.

Enhancement Goals

- Generate a notification when a user raises a new ticket.
- Generate a notification when a ticket status changes (e.g., Open → In Progress, In Progress → Resolved).
- Store notifications in a dedicated database table with ticket reference, type, message, read flag, and timestamp.
- Expose a backend REST API endpoint for the Admin dashboard to fetch unread notifications and notification history.
- Display an unread notification count badge on the Admin dashboard navigation or header.

- Show a notification panel or dropdown listing recent alerts with ticket ID, status, and timestamp.
- Allow administrators to mark notifications as read, clearing the unread badge count.
- Keep all existing ticket creation, status update, and authentication flows unchanged.

Enhancement Summary

Component	Description
Backend	NotificationService + REST controller (Spring Boot)
Frontend	Notification badge + panel in Admin dashboard (React)
Database	admin_notifications table (additive schema change)
Trigger Events	New ticket raised, ticket status changed
API Compatibility	Fully backward compatible — new endpoints only
Existing Flow Impact	None — notification hooks added without modifying existing controllers

---

ENHANCEMENT 5: ADMIN DASHBOARD NOTIFICATIONS

The Admin dashboard is enhanced with a notification system so administrators can immediately see when a new ticket is raised and when ticket statuses are updated, without needing to manually poll or refresh the dashboard.

Notification Trigger Events

- New Ticket Raised: A notification record is created whenever a user submits a new support ticket. The notification includes the ticket ID, the submitting user reference, the initial status (Open), and a timestamp.
- Ticket Status Changed: A notification record is created whenever an administrator or the system updates a ticket's status. The notification captures the ticket ID, the previous status, the new status, and a timestamp.

Figure 4a: Admin dashboard notification panel showing unread notification count badge

Figure 4b: Notification dropdown listing recent ticket events with ticket ID, status, and timestamp

Notification Record Structure

Each notification stored in the admin_notifications table contains the following fields:

Field	Type	Description
id	Long / UUID	Primary key for the notification record
notification_type	Enum / String	NEW_TICKET or STATUS_CHANGED
ticket_id	Long (FK)	Reference to the associated ticket
user_reference	String	Username or user ID who triggered the event
previous_status	String (nullable)	Previous ticket status, null for new tickets

new_status	String	Current ticket status after the event
message	String	Human-readable notification summary
is_read	Boolean	Whether the admin has marked this notification as read
created_at	Timestamp	Time the notification was generated

Backend Implementation

The backend notification logic is implemented in a dedicated NotificationService within the Spring Boot application. Key implementation steps:

1. Created the admin_notifications table via a database migration script. The schema change is purely additive and does not alter any existing tables.
2. Implemented NotificationService with methods createNewTicketNotification() and createStatusChangeNotification(), each accepting the relevant ticket and user context as parameters.
3. Injected NotificationService into the TicketController and called the appropriate method at the end of the ticket creation handler and the ticket status update handler, after the existing business logic completes.
4. Added a GET /api/admin/notifications endpoint that returns all unread notifications ordered by creation time descending, along with the total unread count.
5. Added a PATCH /api/admin/notifications/{id}/read endpoint to mark a single notification as read.
6. Added a PATCH /api/admin/notifications/read-all endpoint for bulk mark-as-read.

Frontend Implementation

The notification panel is implemented in the React Admin dashboard. Key implementation steps:

7. Added a notification bell icon with an unread count badge to the Admin dashboard header or navigation bar.
8. Implemented a useEffect hook that polls GET /api/admin/notifications at a regular interval (e.g., every 30 seconds) to fetch updated notification data.
9. Built a NotificationPanel dropdown component that renders the list of recent notifications with ticket ID, event type, status, and timestamp.
10. On clicking a notification, the system marks it as read via the PATCH endpoint and optionally navigates to the relevant ticket detail view.
11. The unread badge count updates reactively when new notifications arrive or existing ones are marked as read.

Compatibility & Clean Code Notes

- All notification creation calls are appended after existing ticket controller logic completes, ensuring no disruption to current ticket creation or status update behavior.
- NotificationService is a standalone Spring component with no cross-dependencies on authentication, SLA, or email services.
- The admin_notifications table is added via a migration file; no existing table structure is altered.
- The polling interval on the frontend is configurable and chosen to balance responsiveness with server load.
- Notification data is never cached in a way that could show stale read/unread states across admin sessions.

TECHNICAL IMPLEMENTATION PLAN

Backend Module

Aspect	Detail
Service Class	NotificationService.java (new)
Controller	NotificationController.java (new) + hook in TicketController.java
Repository	AdminNotificationRepository.java (Spring Data JPA)
Entity	AdminNotification.java (new JPA entity)
Endpoints Added	GET /notifications, PATCH /notifications/{id}/read, PATCH /notifications/read-all
Trigger Points	TicketController.createTicket() and TicketController.updateStatus()

Database Changes

A single new table is added via migration:

- admin_notifications: stores notification_type, ticket_id, user_reference, previous_status, new_status, message, is_read, created_at.

No existing tables are modified. All changes are additive and migration-driven for predictable deployment across environments.

Frontend Module

Aspect	Detail
Component	NotificationPanel.jsx (new), AdminHeader.jsx (modified to add badge)
Data Fetching	useEffect polling with configurable interval
State	notifications[], unreadCount managed via useState
API Calls	GET /api/admin/notifications, PATCH mark-as-read
UI Elements	Bell icon badge, dropdown panel, per-item read toggle

---

OUTCOME & RESULTS

Features Delivered

Feature	Status	Notes
Notification on New Ticket Raised	**Implemented**	Triggered inside TicketController after ticket creation
Notification on Status Change	**Implemented**	Triggered inside TicketController after status update
admin_notifications DB Table	**Implemented**	Additive migration, no existing tables changed

GET /api/admin/notifications	**Implemented**	Returns unread list with count, ordered by recency
Mark-as-Read API (single & bulk)	**Implemented**	PATCH endpoints for per-item and read-all actions
Unread Badge on Admin Dashboard	**Implemented**	Badge count updates via polling interval
Notification Panel Dropdown	**Implemented**	Lists ticket ID, event type, status, timestamp
Backward API Compatibility	**Maintained**	Existing endpoints and flows unchanged

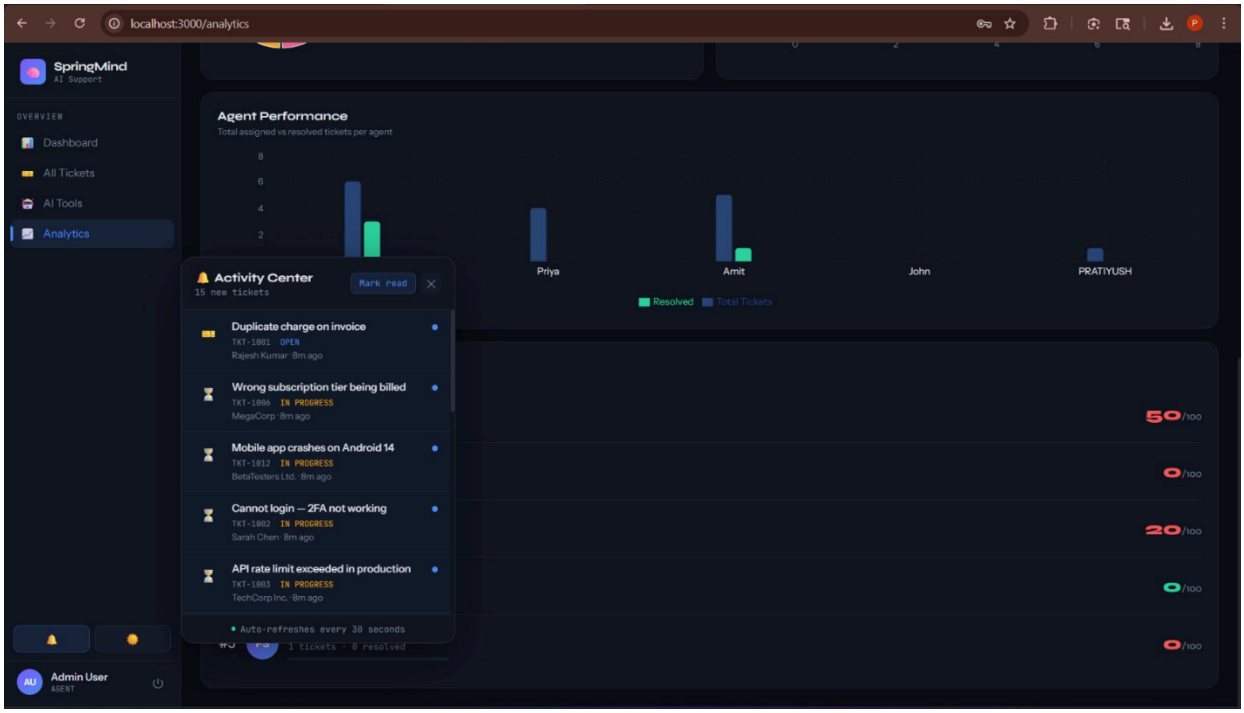

Figure 4a: Admin dashboard notification panel screenshot

Validation Checklist

- A notification record is created in admin_notifications each time a new ticket is submitted.
- A notification record is created each time a ticket status is updated by admin or system.
- Notification records include correct ticket ID, event type, status values, and timestamp.
- GET /api/admin/notifications returns only unread notifications with correct unread count.
- PATCH mark-as-read correctly sets is_read to true for the targeted notification.
- PATCH read-all correctly marks all unread notifications as read in a single call.
- The unread badge on the Admin dashboard reflects the live unread count accurately.
- The notification panel displays ticket ID, notification type, status, and formatted timestamp.
- Existing ticket creation, status update, login, and SLA flows are unaffected by this enhancement.
- Database migration runs cleanly without altering existing tables or data.

CONCLUSION

The Admin Dashboard Notifications enhancement by Neeraj Gupta introduces a proactive alerting layer to the Ticket Management System. By generating and surfacing notifications for new ticket submissions and status transitions, administrators are no longer dependent on manual dashboard refreshes to stay informed of system activity. The unread badge and notification panel provide a familiar, low-friction interface for tracking ticket events in near real-time.

The implementation follows a clean modular design: a dedicated NotificationService handles all creation logic, a separate controller exposes the notification API, and the frontend polling mechanism keeps the dashboard in sync. The additive database migration and non-invasive hook points in existing controllers ensure the enhancement integrates safely with the current system without breaking any existing functionality.

Learning Outcomes

- Design and implementation of an event-driven notification service in Spring Boot.
- Additive database schema design using migration files for safe, predictable deployment.
- REST API design for notification retrieval and read-state management.
- React frontend integration of polling-based data refresh with badge and panel UI components.
- Non-invasive extension of existing controllers by appending notification hooks after core business logic.
- End-to-end feature implementation spanning backend service, REST API, database, and frontend UI.